Lab 3: Multi-Agent Research System

Alignment: Domains 1, 2 and 5 - Orchestration, Tools and Reliability Estimated time: 75 minutes Mode: Local simulation; no API key required

Objective

Delegate bounded research tasks to isolated workers, propagate structured errors, and synthesize claims without losing provenance or uncertainty.

Scenario

An architecture team must compare three deployment controls. Each researcher reads only its assigned source. The coordinator must preserve disagreement instead of blending claims into false certainty.

Prerequisites

- Python 3.10 or newer.
- A terminal opened in this lab folder.
- No production account, network access, or secret is required.

Architecture before execution

Read the source and identify the control boundary, the evidence artifact, and the terminal condition before running it. All side effects are confined to this lab's evidence/ folder.

Procedure

1. Inspect the coordinator task packets and tool grants.``bashpython3 -B research_system.py --show-plan`### 2. Run the full workflow and inspect each worker result envelope.`bashpython3 -B research_system.py`### 3. Open the synthesis and locate source IDs, confidence values, and unresolved conflicts.`bashpython3 -m json.tool evidence/synthesis.json`### 4. Change one source status to **unavailable**, rerun, and observe how the coordinator propagates the retryable error. Restore the fixture afterward.`bashpython3 -B research_system.py --simulate-error`### 5. Run the verifier and save its output.`bashmkdir -p evidence && python3 -B verify.py | tee evidence/verification.txt``

Acceptance checks

[ ] Workers receive different source IDs and no shared conversation history.

[ ] Errors include code and retryability.

[ ] Every claim cites a source ID.

[ ] Conflicts remain visible for human adjudication.

Reflection

Why should the synthesizer preserve a disagreement even when a single fluent answer would be easier to read?

Evidence and completion

Save terminal output in evidence/verification.txt. The lab is complete when the verifier prints PASS, every acceptance check is satisfied, and your reflection identifies one design trade-off.

Troubleshooting

- Run commands from this lab folder so relative paths resolve correctly.
- Use python3 -B to prevent learner machines from creating __pycache__ artifacts.
- If a check fails, read the named failed assertion, inspect the referenced file, and rerun only after correcting the underlying design.

Clean-up

Remove only files you created outside this folder. Never store API keys, access tokens, customer data, or production logs in lab evidence.